

Architectuur- en ontwikkelstrategie

Invaarcalculator™ v3.0.158

IT-architectuur, workflow en validatie-strategie op hoofdlijnen

Innovation Nestor

16 mei 2026

Inhoudsopgave

1	Inleiding	3
2	Architectuur in één pagina	3
2.1	Wat de calculator is	3
2.2	Distributie	3
2.3	Componenten	3
3	Technologie-keuzes	3
3.1	TypeScript voor engine en UI	3
3.2	Zod voor schema-validatie	4
3.3	esbuild voor bundeling	4
3.4	pdfLaTeX voor documenten	4
3.5	Geen externe dependencies in productie	4
3.6	Geen tracking, geen ads, geen cookies	4
4	Codebase-structuur	4
4.1	Scheiding engine en UI	5
5	Ontwikkel-workflow	5
5.1	Sessies als ontwikkelingseenheid	5
5.2	De handoff-discipline	5
5.3	Versie-discipline	6
6	Validatie-strategie	6
6.1	Test-suite	6
6.2	Golden-fixture	6
6.3	Behavior-freeze voor refactors	7
6.4	Jsdome-handmatige checks	7
7	Samenwerking met AI-assistenten	7
7.1	Wat AI-assistenten doen	7
7.2	Waarom dit werkt	7
7.3	Wat de mens doet	7
7.4	Wat dit niet is	7

8	Open-source en licentie	8
9	Wat dit document niet beschrijft	8

1 Inleiding

Dit document beschrijft op hoofdlijnen hoe de Invaarcalculator is opgebouwd en hoe hij wordt ontwikkeld. Het is geschreven voor lezers die de inhoudelijke onderwerpen (Modelspecificatie, Demografie-verantwoording) elders al kennen en willen begrijpen hoe het kale instrument er onder de motorkap uit ziet — welke technologieën gebruikt zijn, hoe nieuwe versies tot stand komen, hoe correctheid wordt geborgd, en hoe de samenwerking met AI-assistenten in de praktijk werkt.

2 Architectuur in één pagina

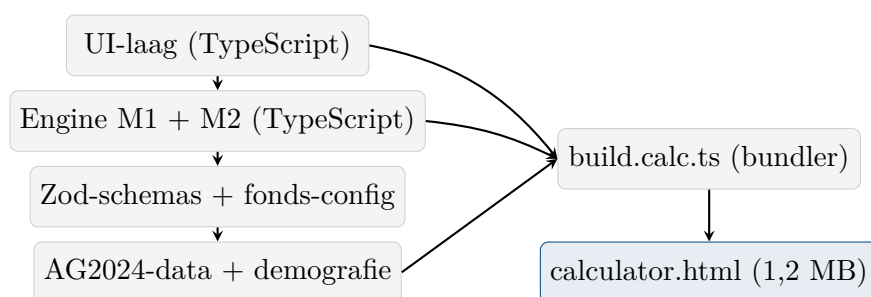
2.1 Wat de calculator is

Eén HTML-bestand, ongeveer 1,2 MB groot, draaiend in elke moderne browser. Geen server, geen database, geen backend. Alle berekeningen voltrekken zich in JavaScript in de browser van de gebruiker; er gaat geen invoer over het netwerk de deur uit. De twee meegeleverde PDF-documenten (Modelspecificatie en Demografie-verantwoording) en de AG2024-sterftafel zitten base64-geëncodeerd in datzelfde HTML-bestand ingebed.

2.2 Distributie

De broncode staat publiek op GitHub (github.com/Invaarcalculator/invaarcalculator). Het gegenereerde HTML-bestand wordt naar de `main`-branch ge-upload als `index.html`; GitHub Pages serveert dat statisch op invaarcalculator.nl. Geen CI/CD-pipeline; geen automatische deployment; updates gebeuren door drag-and-drop van het bestand in de GitHub-webinterface.

2.3 Componenten



Figuur 1: Component-overzicht. De engine-laag voert berekeningen uit; de UI-laag rendert resultaten; de schema-laag valideert de fonds-parameters; de bundler combineert alles tot één bestand.

3 Technologie-keuzes

3.1 TypeScript voor engine en UI

De gehele rekenkern, UI-rendering en build-tooling zijn in TypeScript geschreven. Reden: één taal voor alle code-paden, statisch type-systeem voor compile-time-fouten, en herbruikbaarheid tussen browser-runtime en server-side (Node.js) test-runner. De code is geschreven zonder framework: vanilla DOM-manipulatie, geen React/Vue/Svelte. Voor een single-page app van deze grootte (1.700 regels UI-code) is een framework overhead zonder vergoeding.

3.2 Zod voor schema-validatie

Alle fonds-parameters worden geladen via Zod-schemas. Dat geeft compile-time TypeScript-types en runtime-validatie van de invoer. Bij ongeldige fonds-configuratie faalt de calculator hard bij het laden, niet stilletjes bij een rekenstap. Het schema-bestand (`src/schema.v2.ts`) is ruim 1.000 regels lang en bevat per veld de bronvermelding en de toegestane waarde-range.

3.3 esbuild voor bundeling

De build-stap (`build.calc.ts`) gebruikt esbuild om alle TypeScript-bestanden in één JavaScript-bundel te compileren, voegt de embedded data (AG2024-sterftafel, beide PDF's) toe, en wraps alles in een HTML-template met inline CSS. Resultaat: `calculator.html` als zelfdragend single-file artefact.

3.4 pdfLaTeX voor documenten

De Modelspecificatie en de Demografie-verantwoording (en dit document zelf) zijn pdfLaTeX-bronnen, gecompileerd naar PDF, en als base64-string in het HTML-bestand ingebed. Bij klikken op een knop in de UI wordt de base64 omgezet naar een Blob en als download aangeboden. Reden voor LaTeX: typografische kwaliteit voor formele documentatie, en plaintext-bron die in versiebeheer kan zonder binary-diff-problemen.

3.5 Geen externe dependencies in productie

De gebundelde `calculator.html` bevat geen externe scripts, fonts, of API-aanroepen. Geen Google Fonts, geen jQuery-CDN, geen externe icoon-bibliotheek. Devtime-dependencies (TypeScript, Zod, jsdom voor tests, esbuild) zijn allemaal beperkt tot build- en test-fasen; in de gebruiker-zichtbare HTML zit alleen wat de gebruiker nodig heeft om de calculator te draaien.

3.6 Geen tracking, geen ads, geen cookies

De pagina zet geen cookies, gebruikt geen analytics, en stuurt geen invoer naar een server. De enige netwerk-activiteit is het éénmaal ophalen van `calculator.html` bij paginaload via GitHub Pages. Wat GitHub Pages aan eigen logging doet (IP-adres, User-Agent, tijdstempel) staat buiten de invloed van deze tool en is gedocumenteerd in de privacy-sectie van de UI.

4 Codebase-structuur

Pad	Inhoud
<code>src/engine.m1.ts</code>	Kapitaalwaarde-berekening van de huidige aanspraak (KV), inclusief AG2024-sterftafel-lookups, annuïteitsfactoren, en joint-life-partner-mechaniek met broken-heart-correctie.
<code>src/engine.m2.ts</code>	Persoonlijk pensioenvermogen na invaren (PPV), inclusief fondsbalans-decompositie, cohort-aggregatie voor de fondsschalingsfactor x_{fonds} , en de alternatieve invaarmethode (Ortec-proxy).
<code>src/schema.v2.ts</code>	Zod-schemas voor fonds-parameters en deelnemer-input; runtime-validatie met inline bronvermelding per veld.
<code>src/ui.app.ts</code>	UI-orchestratie: state-management, event-handlers, modal-orchestratie, render-loop.
<code>src/ui.m1.ts</code> , <code>src/ui.m2.ts</code>	UI-componenten per rekenmodule: invoervelden, sliders, KPI-blokken.

Pad	Inhoud
<code>src/ui.sensitivity.ts</code>	Scenario-grafieken (verhoging-vs-indexatie, dekkingsgraad-sweep, bijstort-effect).
<code>src/ui.modal.ts</code> , <code>src/ui.toast.ts</code>	Accessibility-correcte modal-helper en niet-blokkerende toast-meldingen.
<code>src/content/help/*.ts</code>	Tekstuele helpinhoud per parameter, gescheiden van rekencode.
<code>fondsen/sspf/fonds.config.ts</code>	SSPF-specifieke parameter-waarden (fondsvermogen, dekkingsgraad, demografie, etc.).
<code>scripts/run-tests.ts</code>	Test-runner met manifest-gestuurde uitvoering en pass/fail/known-fail-categorisatie.
<code>scripts/snapshot.ts</code>	Sessie-afsluitings-tool: bouwt, zipt, en levert de drie sessie-bestanden.
<code>build.calc.ts</code>	Bundle-script: combineert TypeScript-bundel, AG2024-data en PDF's in één HTML.
<code>fixtures/golden_sspf_default.json</code>	Vastgelegde anker-uitkomsten voor regressie-detectie.

Totale codebase: ongeveer 10.600 regels TypeScript verdeeld over deze bestanden. De engine zelf is ongeveer 77 KB (M1 + M2 samen).

4.1 Scheiding engine en UI

Een bewuste discipline: rekencode en UI-code raken elkaar niet. De engine-bestanden (`engine.m1.ts`, `engine.m2.ts`) zijn pure functies — gegeven dezelfde input leveren ze dezelfde output, zonder DOM-afhankelijkheden, zonder side-effects. De UI-laag bouwt input-objecten op uit DOM-events, roept de engine aan, en rendert het resultaat. Deze scheiding maakt dat de engine los van een browser getest kan worden (engine-tier-tests in Node.js) en dat de UI-laag onafhankelijk gewijzigd kan worden (jsdom-tier-tests).

5 Ontwikkel-workflow

5.1 Sessies als ontwikkelingseenheid

Het werk gebeurt in afgeronde sessies van enkele uren tot enkele dagen. Een sessie begint met het uploaden van drie bestanden uit de voorgaande sessie:

- `Keep-14_v<X.Y.Z>_sessie.zip` — volledige werkdirectory-snapshot
- `HANDOFF_v<X.Y.Z>.md` — wat er gewijzigd is, welke validatie gedaan is, openstaande items
- `calculator_v<X.Y.Z>.html` — gebouwde calculator voor visuele referentie

Een sessie eindigt met dezelfde drie bestanden voor de nieuwe versie. De handoff is altijd leidend voor de actuele status; de zip bevat alle historische handoffs als referentie.

5.2 De handoff-discipline

Elke versie krijgt een eigen handoff-document met daarin:

- Wat is gewijzigd (welke bestanden, welke functies, welke gedragsverandering)
- Welke validatie is gedaan (test-resultaten, jsdom-verificatie, eventuele numerieke vergelijking)

- Open punten of vervolg-aanbevelingen
- Versie-bump-spoor: `src/ui.app.ts` `RELEASE_VERSION`, `package.json` `version`, en — bij engine-wijzigingen — `fixtures/golden_sspf_default.json` `release_anchor`

Het snapshot-script faalt hard als de bijbehorende handoff ontbreekt. Dat is bewust: een versie zonder handoff is niet vindbaar bij toekomstige sessies, en zou de continuïteit van de samenwerking met AI-assistenten breken.

5.3 Versie-discipline

Soort wijziging	Versie-bump	Handoff-naam
Engine, UI, build-output	Ja	HANDOFF_v<X.Y.Z>.md
Tooling (scripts, tests, docs)	Nee	HANDOFF_v<X.Y.Z>-tooling.md

Versie-nummers volgen het patroon `3.0.X`: major en minor zijn stabiel op 3 en 0, patch-nummer telt sessies. Een typische sessie verhoogt het patch-nummer met 1 (één commit) tot enkele eenheden (meerdere onderwerpen in één sessie). Major/minor-bumps zijn gereserveerd voor structurele wijzigingen aan engine of schema.

6 Validatie-strategie

6.1 Test-suite

Bij elke sessie-afsluiting draait een volledige test-run. Het manifest in `scripts/run-tests.ts` stuurt circa elf test-bestanden in twee tiers:

- **Engine-tier**: pure TypeScript-tests draaiend in Node.js, geen browser nodig. Onder andere KV-smoke-tests, M2-cascade-tests, alternatieve-methode-consistentie-tests, build-portability-tests en *golden-fixture-test* (zie hieronder).
- **Jsdom-tier**: tests die de gebouwde HTML in een gesimuleerde browser laden en interactief gedrag verifiëren — aftrek-komma-input, browser-calc-end-to-end, modal-toegankelijkheid, toast-gedrag, extensie-noise-filter.

De typische verwachte uitkomst: 10 tests “pass”, 1 test “known-fail” (AG2024 vs Gompertz-vergelijking; structureel, in een toekomstige versie te adresseren). Een rode uitslag op één van de 10 pass-tests blokkeert de sessie-afsluiting.

6.2 Golden-fixture

Het bestand `fixtures/golden_sspf_default.json` bevat de exacte verwachte engine-uitkomst voor het UI-default-scenario (70-jarige man, EUR 100.000 pensioen, geen partner, SSPF-defaults). Bij elke engine-wijziging wordt deze fixture gecontroleerd: een afwijking van meer dan een fractie van een procent op KV, PPV of intermediate-grootheden vereist een expliciete update van het fixture-bestand mét bevestiging van de nieuwe “ankerwaarde”.

Dit beschermt tegen drift: kleine code-wijzigingen die per ongeluk een numerieke output verschuiven, slaan stuk op de golden-fixture-test en moeten dan verantwoord worden in de handoff voordat de fixture wordt geactualiseerd.

6.3 Behavior-freeze voor refactors

Bij refactor-stappen (opschoning, herstructurering, dode-code-verwijdering) gebruiken we een zwaardere check: de gebundelde JavaScript-output wordt byte-voor-byte vergeleken met de vorige versie. Identiek (op de versie-string na) = bewezen geen gedragsverandering. Een refactor die de bundel-bytes onveranderd laat, is per definitie gedrag-neutraal en kan zonder verdere numerieke verificatie aanvaard worden.

6.4 Jsdm-handmatige checks

Voor UI-wijzigingen die het normale test-suite niet exhaustief dekt (nieuwe knoppen, modal-volgorde, dropdown-gedrag), schrijven we per sessie ad-hoc jsdom-scripts die de gebouwde HTML laden en de specifieke wijziging verifiëren. Deze scripts staan niet in versie-controle — ze zijn wegwerp-instrumenten voor éénmalige verificatie en worden in de handoff genoteerd onder “Verificatie”.

7 Samenwerking met AI-assistenten

7.1 Wat AI-assistenten doen

De codebase is iteratief tot stand gekomen in dialoog met meerdere AI-assistenten: voornamelijk Anthropic Claude (diverse versies, op het moment van schrijven Claude Opus 4.7) voor TypeScript-implementatie, modelspecificatie-redactie en architectuurkeuzes; PensioenGPT voor domeinspecifieke navigatie van Pensioenwet en bijlagen; aanvullende assistenten voor kruiscontrole en onderzoeksvragen.

7.2 Waarom dit werkt

Drie ontwerpkeuzes maken de samenwerking met AI-assistenten praktisch beheersbaar:

- **Pure functies in de engine.** Engine-code zonder side-effects is testbaar en reviewbaar zonder uitvoer-context; een AI-assistent kan een functie lezen, een test schrijven en de uitkomst beredeneren, allemaal op tekst-niveau.
- **Zware schema-validatie.** Zod-fouten zijn expliciet en bevatten de context (welk veld, welke regel, welke waarde). Een AI-assistent die per ongeluk een verkeerd type-veld opstelt, krijgt direct een leesbare foutmelding terug die in de volgende iteratie de fix oplevert.
- **Handoff-conventie.** Elke sessie sluit af met een handoff die de volgende sessie heropent zonder context-verlies. De handoff is leesbaar voor zowel mens als AI-assistent — ze beschrijft niet alleen *wat* gewijzigd is maar ook *waarom*, zodat de volgende iteratie niet opnieuw door dezelfde argumenten hoeft.

7.3 Wat de mens doet

De rolverdeling in een sessie: de gebruiker (auteur) bepaalt richting en doel — welk probleem, welke prioriteit, welke compromissen acceptabel zijn. De AI-assistent implementeert, test en verifieert, met de gebruiker als reviewer en eindbeslissers. Bij elke substantiële keuze (architectuur, terminologie, modelaannamen) levert de AI-assistent meerdere opties met afwegingen; de gebruiker selecteert.

7.4 Wat dit niet is

De AI-assistenten zijn geen autonomous-agents die zelf code committen. Elke wijziging gaat door dezelfde sessie-cyclus: handoff lezen, wijziging voorstellen, code maken, tests draaien, verificatie

tonen, handoff schrijven, snapshot leveren. De gebruiker downloadt de drie bestanden en kiest of hij ze publiceert door `index.html` in GitHub te plaatsen.

Aan de tool kunnen geen rechten worden ontleend; de inhoudelijke verantwoordelijkheid voor wat erin staat ligt bij de auteur, niet bij de AI-assistenten. Aanwezigheid van fouten — in interpretatie van de Pensioenwet, in actuariële formules, in code, of in numerieke uitkomsten — kan niet worden uitgesloten en is op een aantal punten statistisch waarschijnlijk.

8 Open-source en licentie

De volledige broncode is publiek beschikbaar onder CC BY-NC 4.0 (Creative Commons Naamsvermelding-NietCommercieel 4.0 Internationaal). Dat betekent dat onafhankelijke verificatie van de berekeningen mogelijk is, dat afgeleide werken zijn toegestaan met bronvermelding, en dat commercieel gebruik expliciete toestemming vereist.

De documentatie (Modelspecificatie, Demografie-verantwoording, dit document) valt onder dezelfde licentie. Alle drie zijn als plaintext LaTeX-bron in de codebase aanwezig zodat correcties zichtbaar zijn in versie-controle en niet in een binary-format zitten.

9 Wat dit document niet beschrijft

- **De rekenformules zelf.** Die staan in de Modelspecificatie. Dit document beschrijft alleen de *infrastructuur* eromheen.
- **De cohort-data.** Bron, aging-methodiek en M/V-uitsplitsing van de SSPF-cohortbezetting staan in de Demografie-verantwoording.
- **Specifieke implementatie-details.** Voor architectuur op functie-niveau: zie de inline-commentaar in de TypeScript-bestanden zelf en het bestand `ENGINE_SPECIFICATIE_v1.md` in de codebase (normatief specificatie-document, niet primair gebruikersgericht).
- **Roadmap of geplande wijzigingen.** Per-sessie-status staat in de meest recente handoff; een centrale roadmap-tracker bestaat niet.

Einde architectuur- en ontwikkelstrategie.